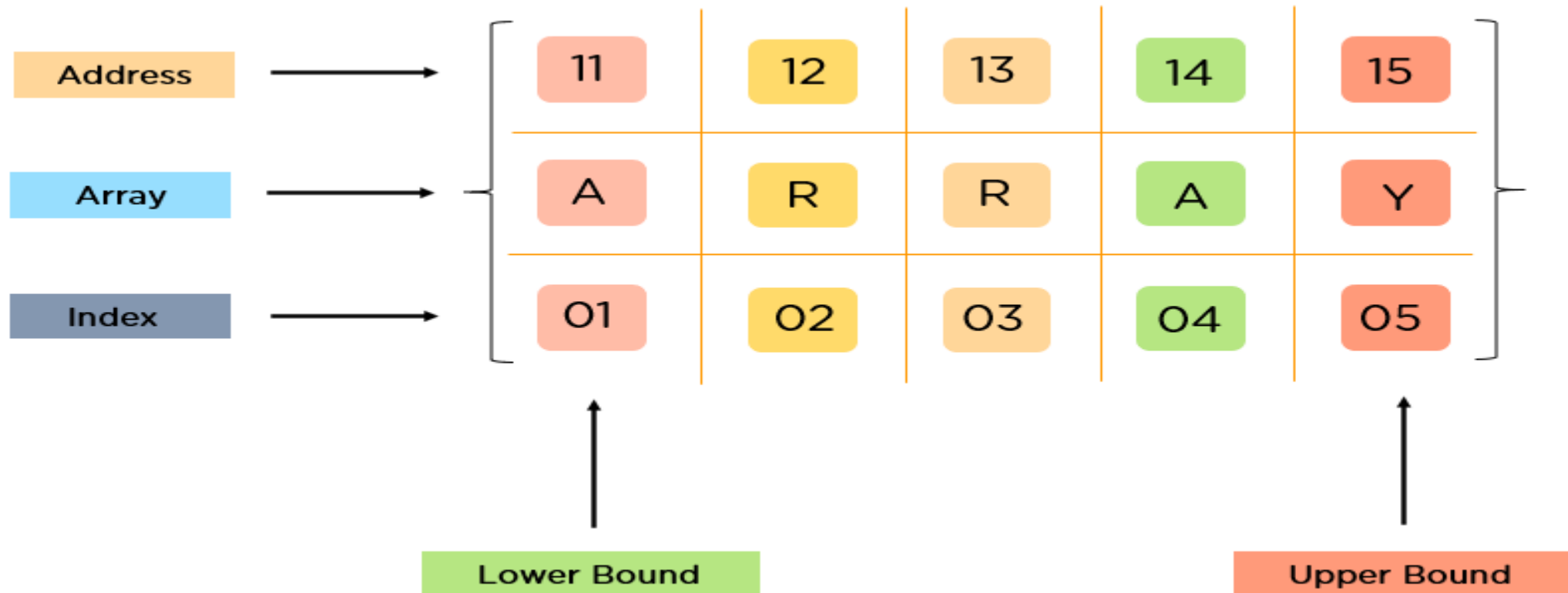


ARRAY & OPERATIONS IN LINEAR ARRAY

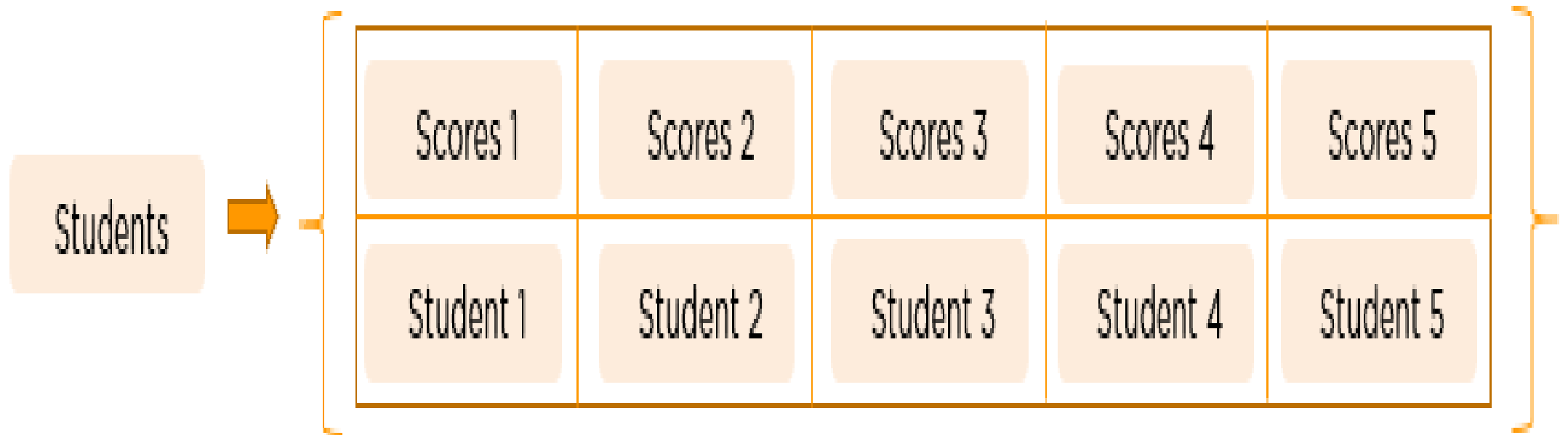
Aditya Tiwari
Assistant Professor
CSVТУ BHILAI

- What Are Arrays in Data Structures?
- An array is a linear data structure that collects elements of the same data type and stores them in contiguous and adjacent memory locations. Arrays work on an index system starting from 0 to $(n-1)$, where n is the size of the array.
- It is an array, but there is a reason that arrays came into the picture.

What Are Arrays in Data Structures?



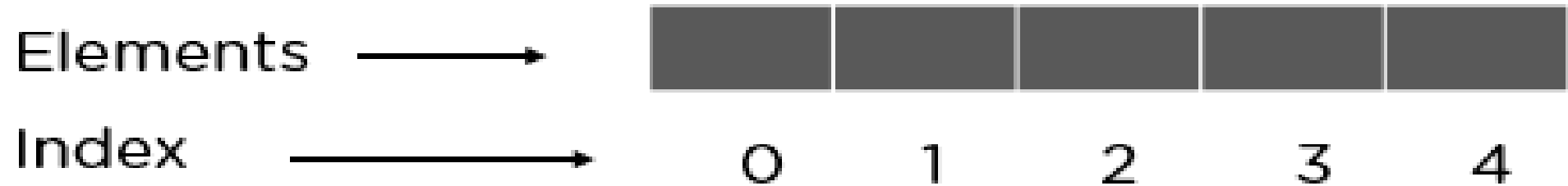
Why Do You Need an Array in Data Structures?



- Let's suppose a class consists of ten students, and the class has to publish their results. If you had declared all ten variables individually, it would be challenging to manipulate and maintain the data.
- If more students were to join, it would become more difficult to declare all the variables and keep track of it. To overcome this problem, arrays came into the picture.

What Are the Types of Arrays?

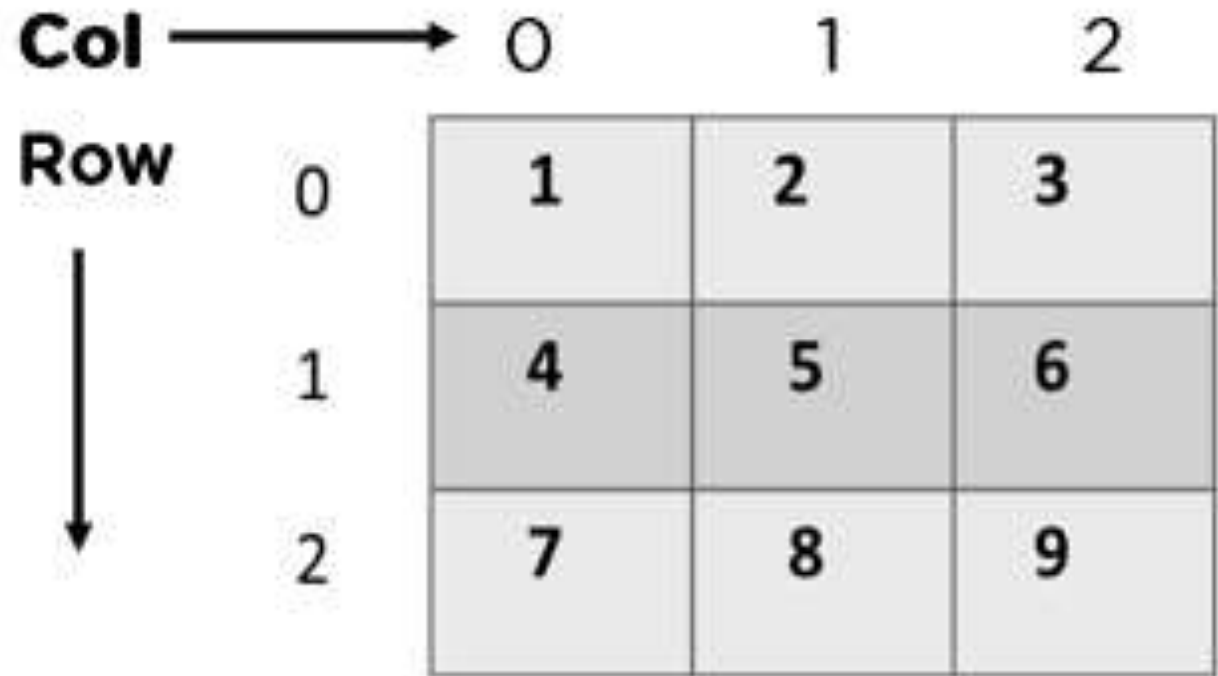
- There are majorly two types of arrays, they are:
- One-Dimensional Arrays:
- You can imagine a 1d array as a row, where elements are stored one after another.



Multi-Dimensional Arrays:

These multi-dimensional arrays are again of two types.
They are:

- Two-Dimensional Arrays:
- You can imagine it like a table where each cell contains elements



The diagram illustrates a 2D array as a table. To the left of the table, the word **Col** is positioned above a horizontal arrow pointing to the column indices (0, 1, 2). Below **Col**, the word **Row** is positioned to the left of a vertical arrow pointing down to the row indices (0, 1, 2). The table itself has three rows and three columns, with cells containing numbers 1 through 9 in a sequential order starting from the top-left.

Col	→	0	1	2
Row	0	1	2	3
	1	4	5	6
	2	7	8	9

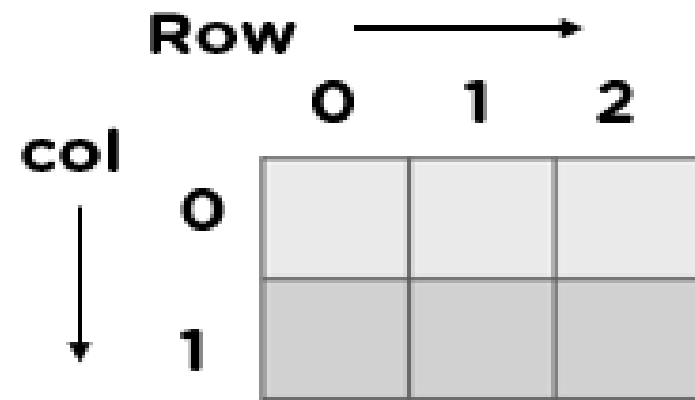
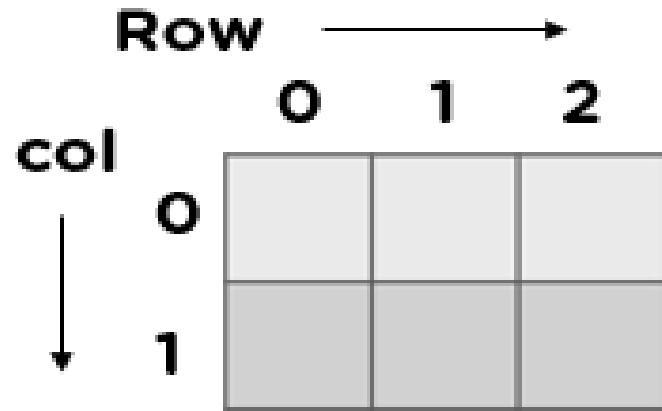
Three-Dimensional Arrays:

You can imagine it like a cuboid made up of smaller cuboids where each cuboid can contain an element.

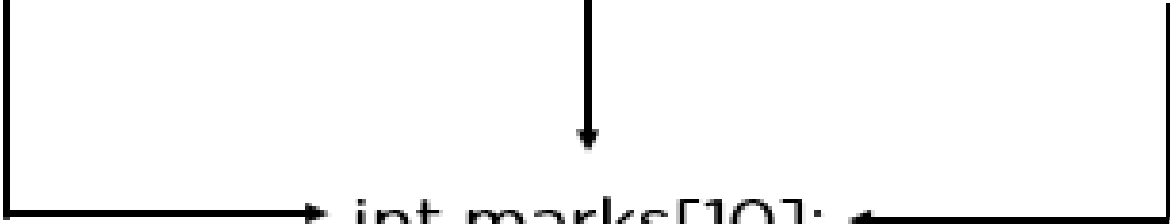
int num[2][3][2];

num[row][col][0]

num[row][col][1]



How Do You Declare an Array?

Array data type	Array Name	Array Size
		
<code>int marks[10];</code>		

Arrays are typically defined with square brackets with the size of the arrays as its argument.

Here is the syntax for arrays:

- 1D Arrays: `int arr[n];`
- 2D Arrays: `int arr[m][n];`
- 3D Arrays: `int arr[m][n][o];`

How Do You Initialize an Array?

You can initialize an array in four different ways:

- Method 1: `int a[6] = {2, 3, 5, 7, 11, 13};`
- Method 2: `int arr[] = {2, 3, 5, 7, 11};`
- Method 3: `int n;`
`scanf("%d",&n);`
`int arr[n];`
`for(int i=0;i<5;i++)`
`{`
`scanf("%d",&arr[i]);`
`}`

- Method 4:

- `int arr[5];`

- `arr[0]=1;`

- `arr[1]=2;`

- `arr[2]=3;`

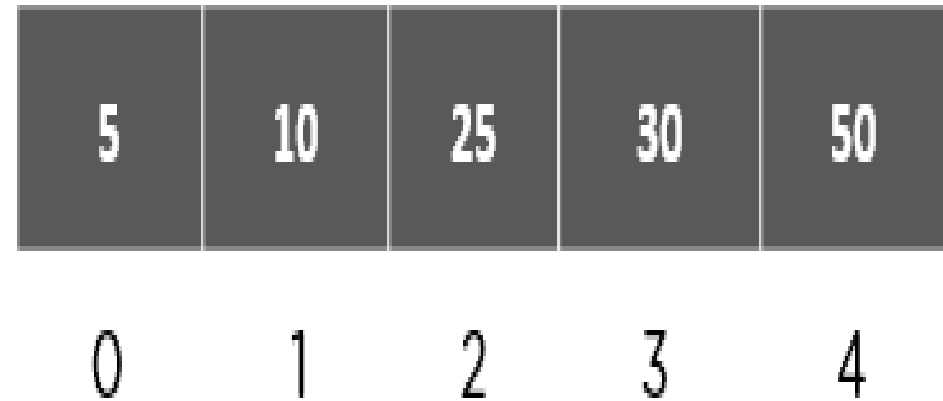
- `arr[3]=4;`

- `arr[4]=5;`

How Can You Access Elements of Arrays in Data Structures?

- You can access elements with the help of the index at which you stored them. Let's discuss it with a code:

- `#include<stdio.h>`
- `int main()`
- `{`
- `int a[5] = {2, 3, 5, 7, 11};`
- `printf("%d\n",a[0]); // we are accessing`
- `printf("%d\n",a[1]);`
- `printf("%d\n",a[2]);`
- `printf("%d\n",a[3]);`
- `printf("%d",a[4]);`
- `return 0;`
- `}`



```
2
3
5
7
11
-----
Process exited after 0.1476 seconds with return value 0
Press any key to continue . . .
```

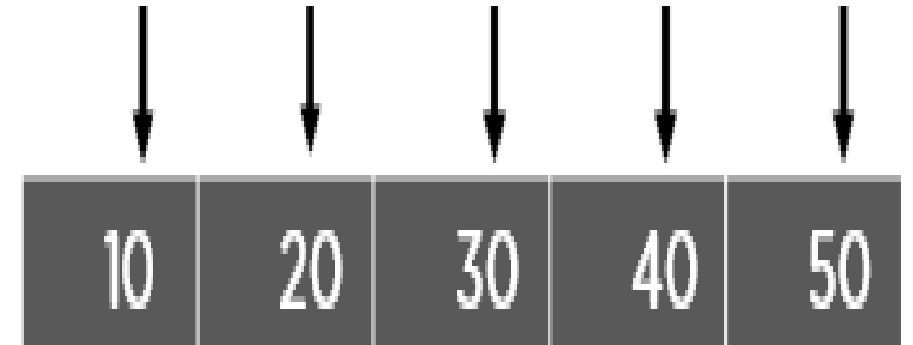
What Operations Can You Perform on an Array?

- Traversal
- Insertion
- Deletion
- Searching
- Sorting

Traversing the Array:

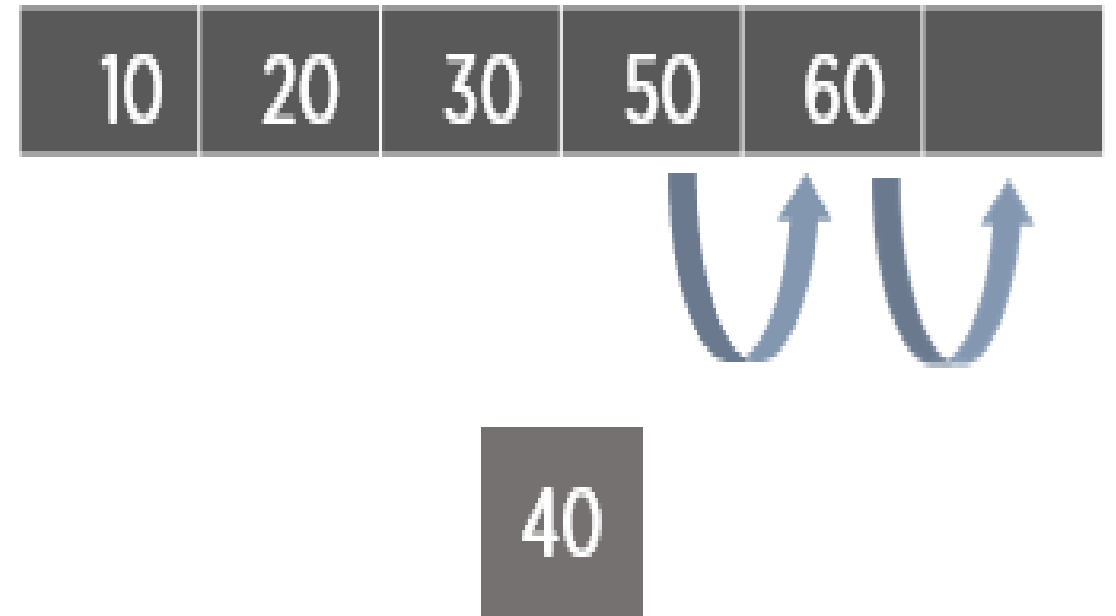
Traversal in an array is a process of visiting each element once.

- Here LA is a linear array with lower bound LB and upper bound UB.
- **Step 1:** Set $K=LB$ [Initialize counter]
- **Step 2:** Repeat step 3 and 4 while $K \leq UB$ [Loop]
- **Step 3:** Apply process to $LA[K]$. [visit element]
- **Step 4:** Set $K=K+1$. [Increase counter]
- [End of step 2 loop]

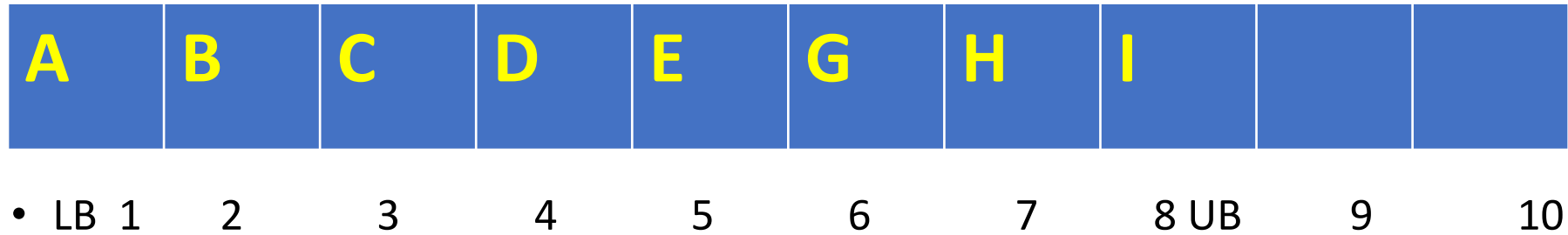


Insertion into an Linear Array

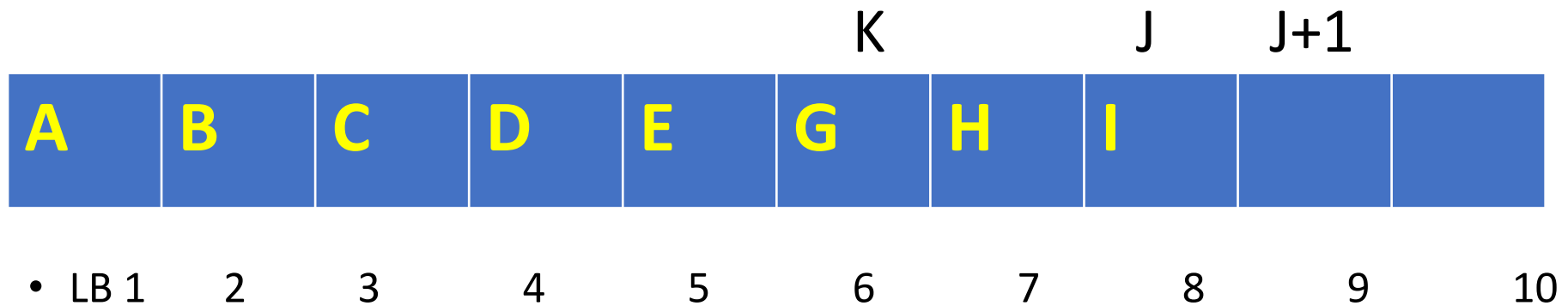
- INSERT (LA,N,K,ITEM)
- Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an elements ITEM into the Kth position in LA.



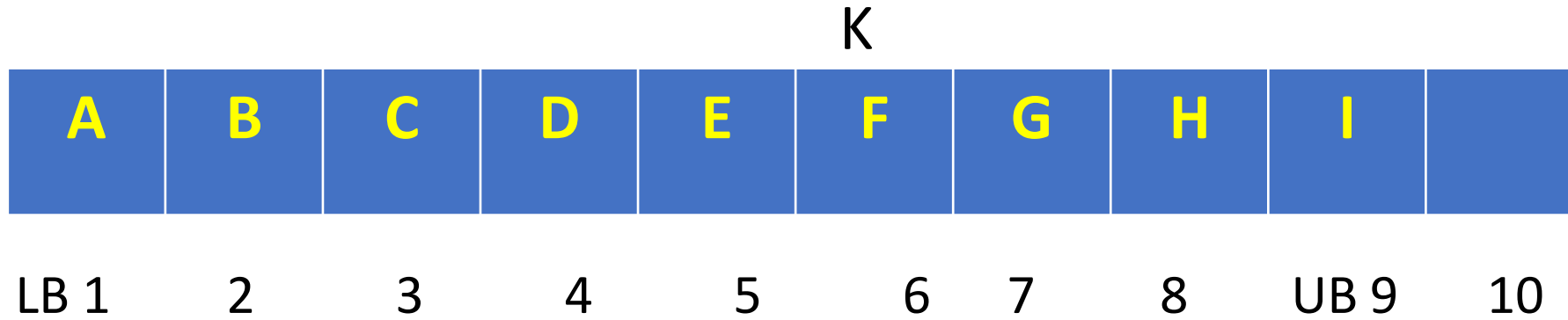
- Example :
- Step 1: Given array with elements



- Step 2: Find Kth position and initialized pointer



- Step 3: Inset F into Kth position



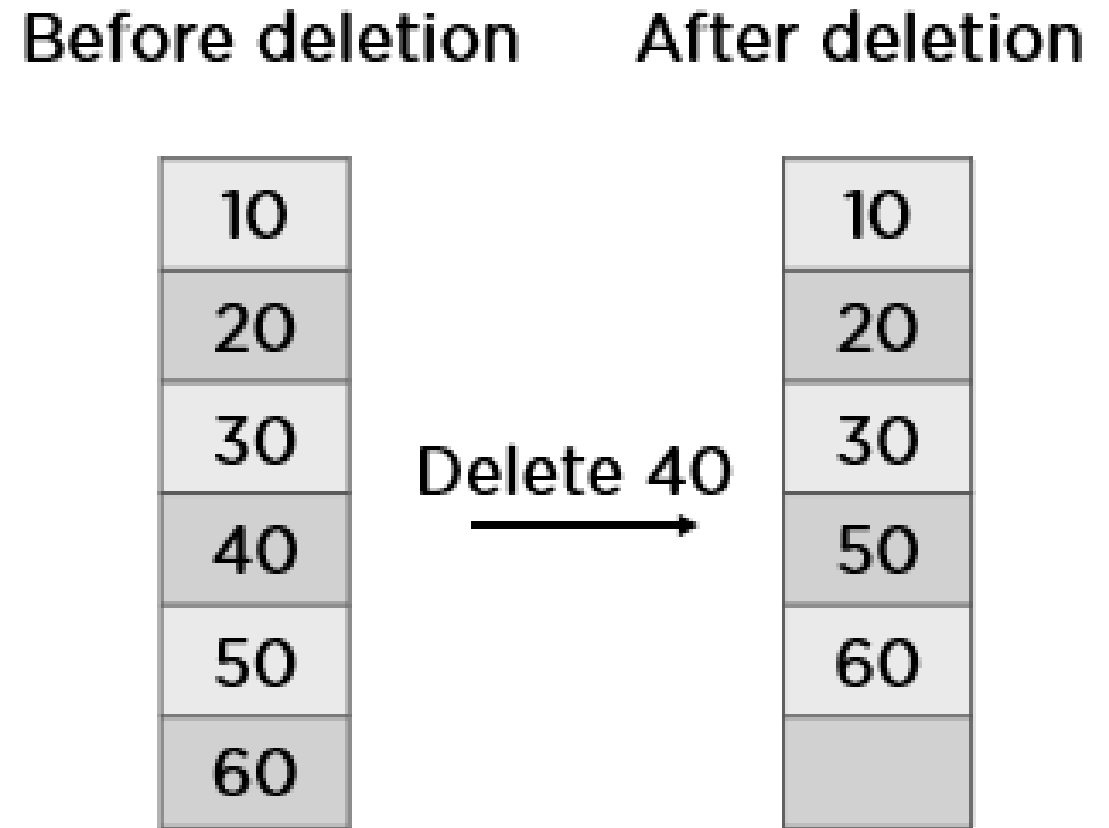
- Insertion in an array is the process of including one or more elements in an array.
- Insertion of an element can be done:
 - ☐ At the beginning
 - ☐ At the end and
 - ☐ At any given index of an array.

- PSEUDO CODE:- FOR INSERTION

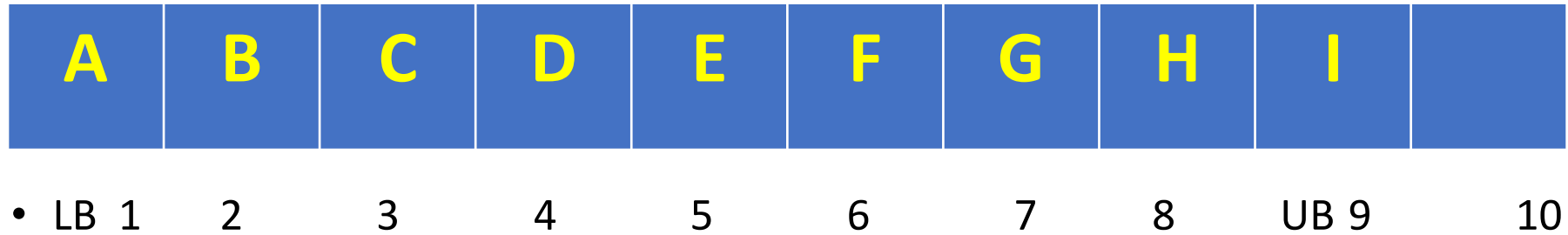
- Step 1: Set $J = N$
- Step 2: Repeat step 3 and 4 while $J \geq K$.
- Step 3: Set $LA [J+1] = LA [J]$ [Move Jth elements downwards]
- Step 4: Set $J = J - 1$. [Decrease counter]
- [End of step 2 loop]
- Step 5: Set $LA [K] = ITEM$
- Step 6: Set $N = N + 1$ [Reset N]
- Step 7: Exit.

Deletion: IN A LINEAR ARRAY

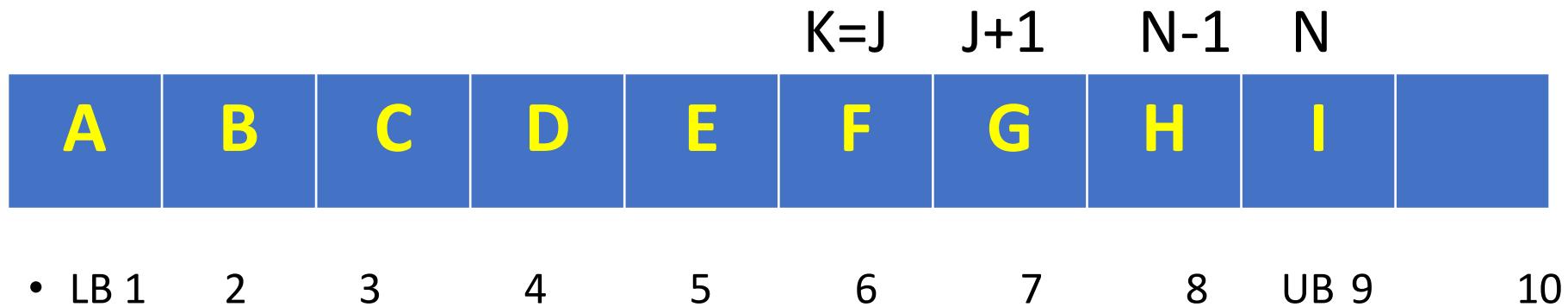
- Deletion of an element is the process of removing the desired element and re-organizing it.
- You can also do deletion in different ways:
 - At the beginning
 - At the end
 - At any Position element



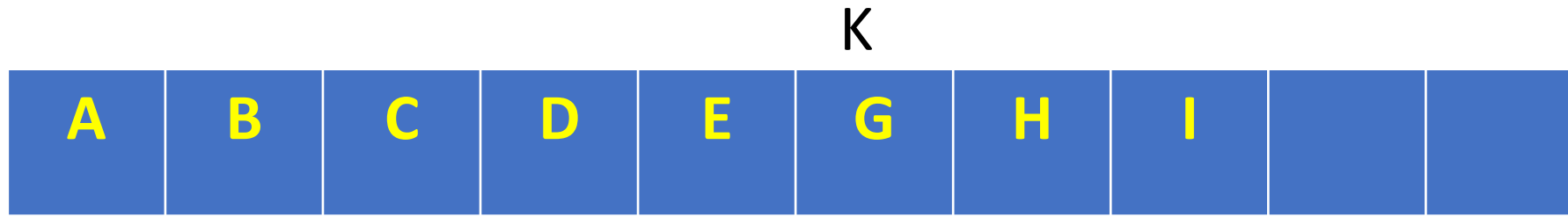
- Example :
- Step 1: Given array with elements



- Step 2: Find Kth position and initialized pointer



- Step 3: Inset F into Kth position



- LB 1 2 3 4 5 6 7 UB 8 9 10

DELETION PSEUDO CODE:-

- Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes the Kth elements from LA.
- Step 1: Set Item = LA [K]
- Step 2: Repeat for J= K to N – 1
 - Set LA [J] = LA [J + 1] [Move J + 1st element upword]
 - [End of loop]
- Step 3: Set N = N – 1. [Reset N]
- Step 4: Exit.

•THANK YOU